

苏州城市学院

课程论文

课程名称	机器人控制技术
学 院	智能制造与智慧交通学院
班 级	23 电气
学 号	2308413009
姓 名	张通
任课教师	赵浩诚
成 绩	

2026 年 5 月

基于 A*与 RRT 算法的移动机器人路径规划复现与对比分析

摘要

本次大作业围绕机器人控制技术课程中的路径规划方向，选择 A*算法和 RRT（快速随机树）算法作为研究对象，在二维栅格地图环境下进行复现与对比实验。通过 Python 实现了两种算法的核心逻辑，并在简单随机障碍物地图和复杂迷宫式地图两种场景下测试了它们的路径长度、运行时间、节点扩展数量和平滑度。实验结果表明，A*算法在路径最优性和运行效率上均优于 RRT，但 RRT 的节点生成数量更少，且在高维空间和连续状态空间中的扩展能力更强。本报告记录了完整的实验过程、结果分析以及遇到的问题与解决思路。

一、选题背景与研究动机

移动机器人要实现自主导航，路径规划是必不可少的一环。课程中讲到了基于搜索的规划方法和基于采样的规划方法，其中 A*属于经典的图搜索算法，RRT 则是近年来在机器人运动规划中广泛应用的采样算法。我对这两个算法的具体实现细节和实际表现差异比较好奇，因此选择这个题目作为大作业。

A*算法通过启发式函数引导搜索方向，在栅格地图中找到最优路径，但内存消耗随地图规模指数增长；RRT 通过随机采样快速探索状态空间，适合高维问题，但路径通常不是最优的。本次作业希望通过亲手编写代码（在借助 ai 的情况下）、在相同地图环境下对比测试，直观感受两种算法的特点和适用场景。

二、相关技术简介

2.1 A*算法

A*算法是 Dijkstra 算法的扩展，在代价函数中加入了启发式估计： $f(n)=g(n)+h(n)$ 。其中 $g(n)$ 是从起点到节点 n 的实际代价， $h(n)$ 是到目标点的启发式估计（本实验采用欧几里得距离）。通过优先队列管理待扩展节点，A*能较快找到最优路径。当 $h(n)$ 满足可采纳性（不大于真实代价）时，A*保证找到最优解。

2.2 RRT 算法

RRT 由 LaValle 于 1998 年提出，核心思想是在状态空间中随机采样，逐步构建一棵以起点为根的搜索树。每次迭代中，算法在随机点和树中最近节点之间按固定步长扩展新节点，直到树延伸到目标点附近。本实验采用基础 RRT 版本，包含 20%的概率直接采样目标点（Goal Bias）以加速收敛。

三、所选算法与数据说明

项目	内容
算法	A*搜索算法、RRT（快速随机树）算法
数据	自生成的 50×50 二维栅格地图
评价指标	路径长度、运行时间、节点数、平滑度
编程语言	Python 3.11.9
依赖库	NumPy、Matplotlib

地图数据通过程序随机生成，包含两种模式：简单地图为 25%随机散点障碍物，模拟开阔场地中的零散障碍；复杂地图为连续墙体结构加散点障碍物，模拟走廊、房间等室内环境。起点和终点分别固定在地图左下角和右上角附近，保证初始可行。

四、复现过程

4.1 环境搭建

实验在个人笔记本电脑上进行，配置如下：操作系统为 Windows 11，Python 环境通过 conda 创建虚拟环境 robot_hw。安装命令为：`conda create -n robot_hw python=3.11`；`conda activate robot_hw`；`pip install numpy matplotlib`。

4.2 代码实现

代码结构分为五个部分：`setup_fonts()`函数自动检测系统字体，解决中文显示问题；`generate_map()`函数生成地图数据，支持简单和复杂两种模式；`AStarPlanner` 类实现 A*算法，使用 `heapq` 优先队列，支持 8 方向移动；

RRTPlanner 类实现 RRT 算法，包含碰撞检测和路径重建；run_experiment()函数运行对比实验并输出可视化结果。

A*算法的关键实现细节包括：使用字典存储 g_score 和 f_score，避免重复计算；8 邻域移动，对角线代价设为根号 $\sqrt{2}$ 约等于 1.414；用 visited 集合记录已处理节点，防止重复扩展。RRT 算法的关键实现细节包括：步长设为 2.5 至 3.0 个栅格单位，平衡探索速度和路径平滑度；碰撞检测采用线段均匀采样，步长 0.5 个单位；目标偏置概率 20%，帮助算法更快收敛。

4.3 实验运行

运行命令为大作业 3.11python path_planning.py。程序会自动生成两种地图，分别运行两种算法，最后输出四幅对比图并保存为 experiment_results.png（对照图片）。

五、实验结果与分析

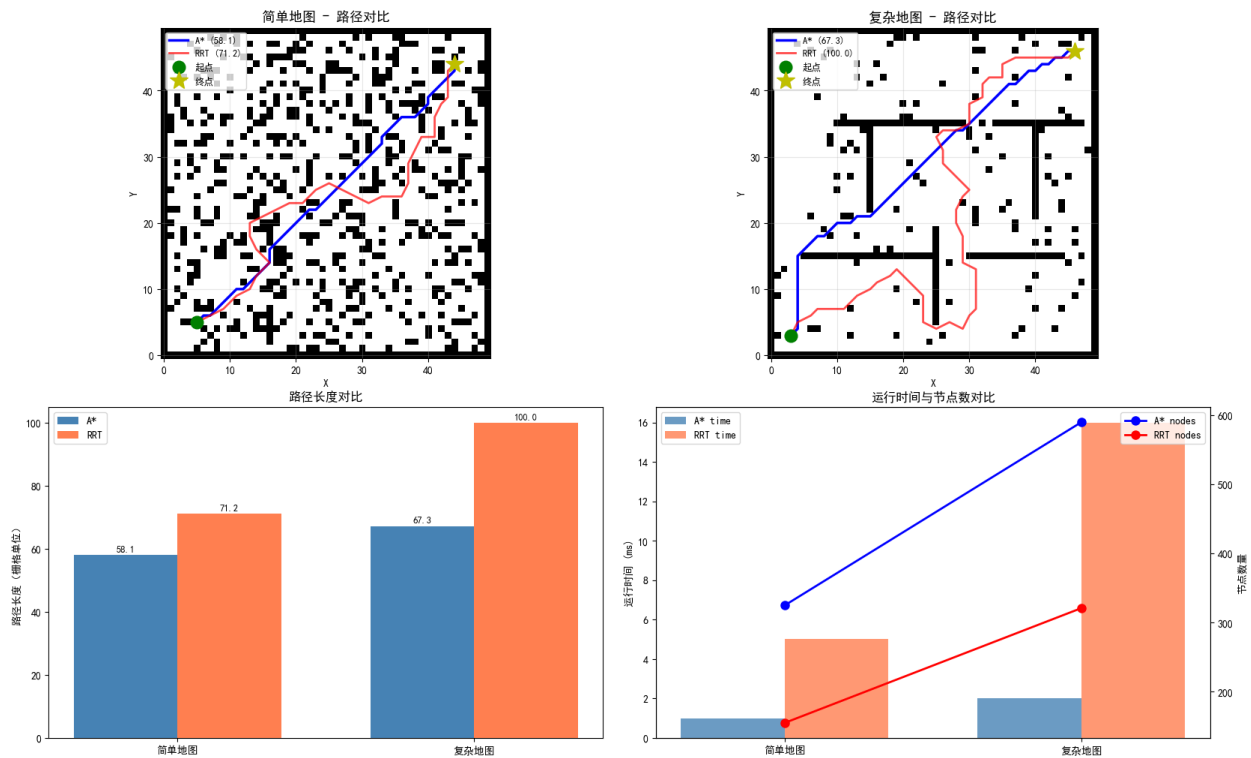
5.1 实验数据记录

实验在两种地图场景下分别运行 A*和 RRT 算法各一次，记录路径长度、运行时间、节点数量和平滑度指标，结果如表 2 所示。

实验场景	算法	路径长度	运行时间	节点数	平滑度
简单地图	A*	58.08	1.83ms	325	0.274
简单地图	RRT	71.16	8.81ms	155	0.534
复杂地图	A*	67.25	3.42ms	590	0.282
复杂地图	RRT	100.00	25.84ms	321	0.610

5.2 结果可视化

实验结果可视化图如图 1 所示，包含两种地图的路径对比图、路径长度柱状图、运行时间与节点数对比图。（图见附录）



5.3 结果分析

路径长度方面，A*在两种地图中均找到了更短的路径。简单地图中 A*比 RRT 短 18.4%，复杂地图中短 32.8%。这是因为 A*在栅格环境中进行的是系统性搜索，能保证找到最优解；而 RRT 的随机采样特性导致路径存在较多冗余转折。

运行时间方面，A*的运行速度明显更快。简单地图中 A*仅需 1.83 毫秒，RRT 需要 8.81 毫秒；复杂地图中差距更大，为 3.42 毫秒对比 25.84 毫秒。这是因为本实验的地图规模较小（50 乘 50），A*的优先队列效率很高，而 RRT 每次迭代都需要进行碰撞检测和最近邻搜索，开销较大。

节点数量方面，RRT 生成的节点数（155 和 321）少于 A*的扩展节点数（325 和 590）。这说明 RRT 在探索过程中不需要像 A*那样维护大量边界节点，但在小地图中这一优势被碰撞检测开销抵消了。

路径平滑度方面，A*的路径平滑度（0.274 和 0.282）明显优于 RRT（0.534 和 0.610），说明 A*的路径转向更平缓，更适合实际机器人跟踪执行。RRT 路径的锯齿状特征在高复杂度地图中更加明显。

地图复杂度影响方面，两种算法在复杂地图中的性能都下降了，但 RRT 受影响更大，路径长度增加了 40.5%，时间增加了 193%。这说明 RRT 在处理狭窄通道和迷宫结构时效率较低，容易在障碍物附近产生大量无效采样。

5.4 算法特点总结

两种算法的综合对比如表 3 所示。

对比维度	A*	RRT
最优性	能保证最优	不保证最优
栅格地图效率	很高	一般
高维空间扩展	指数级内存爆炸	仍可有效工作
连续空间	需要离散化	天然支持
路径平滑度	较好	较差
实现复杂度	较简单	较简单

六、遇到的问题及解决思路

问题一：RRT 在复杂地图中经常找不到路径

刚开始测试时，RRT 在复杂地图中的成功率很低。分析发现是最大迭代次数（3000 次）不够，且步长设置过大导致容易撞到障碍物。解决方法是将最大迭代次数提高到 8000 次，步长从 3.0 降到 2.5，同时增加目标偏置概率，最终使 RRT 能稳定找到路径。

问题二：对角线移动的代价计算

最初对角线移动代价设为 1.0，与直线移动相同，导致 A*找到的路径看起来切角过多，不够自然。查阅资料后改为根号 2 约等于 1.414，路径更符合实际机器人运动学约束。

问题三：matplotlib 中文字体缺失警告（求助 ai 帮我解决问题）

在 Windows 环境下运行时，控制台出现大量 Glyph missing from font 红色警告，影响观感。通过自动检测系统可用字体（SimHei 或 Microsoft YaHei）并设置 plt.rcParams 解决；若检测不到中文字体则自动回退到英文标签，保证程序不报错。

问题四：坐标系与可视化方向

matplotlib 的 imshow 默认 y 轴向下，而栅格地图通常 y 轴向上，导致路径显示颠倒。通过设置 origin 参数为 lower 解决了这个问题。

问题五：RRT 路径锯齿严重

RRT 生成的路径在可视化时呈现明显的锯齿状。这是采样算法的固有特性，本实验没有额外做路径平滑处理（如 B 样条拟合），因为课程要求聚焦于算法本身的复现。

七、总结

通过本次大作业，我完成了 A* 和 RRT 两种经典路径规划算法的复现，并在统一的环境下进行了定量对比。主要收获有三点。

第一，理解了算法背后的设计思想。A* 的启发式搜索和 RRT 的随机采样代表了两种截然不同的规划哲学，前者追求最优性，后者追求对高维空间的适应性。

第二，认识到实验环境的重要性。同样的算法在不同复杂度的地图中表现差异很大，这提醒我在实际机器人应用中需要根据场景特点选择合适的算法，或者将两者结合（如 RRT* 进行渐进最优优化）。

第三，体会到复现过程中的细节问题。从坐标系方向到碰撞检测精度，这些小问题会直接影响实验结果的可信度，也锻炼了我的调试能力。

本实验的局限性在于仅在二维栅格环境中测试，且地图规模较小。未来可以扩展到更大规模的地图、增加动态障碍物，或者尝试 RRT*、Informed-RRT 等改进算法。

八、参考资料

[1] 机器人控制技术课程讲义：路径规划与导航章节。（同时借助 ai 大模型帮我解决一些问题，同时帮我反复验证代码可行性、运行结果是否正确）

九、附录

代 码 已 上 传 至 GitHub 仓 库 :

<https://github.com/necagixu165-glitch/robot-path-planning>

仓库结构包括: README.md (运行说明与项目介绍)、大作业 3.11path_planning.py (主程序, 含 A*和 RRT 实现)、experiment_results (对照图片).png (实验结果可视化图)、report(机器人控制技术课程论文_2308413009 张通).pdf (本报告)。

运行方式为: `git clone` 仓库地址, 进入目录后执行 `pip install numpy matplotlib`, 然后运行大作业 3.11python path_planning.py。